

COMP3320/6464 Week 6, Lecture 2

Rhys Hawkins

Semester 2, 2024

Lecture Plan

Yesterday

- Matrix Transpose
- Loop unroll and jam
- Blocking algorithms

Today

- Matrix Multiply
- Last minute Assignment 1 Questions

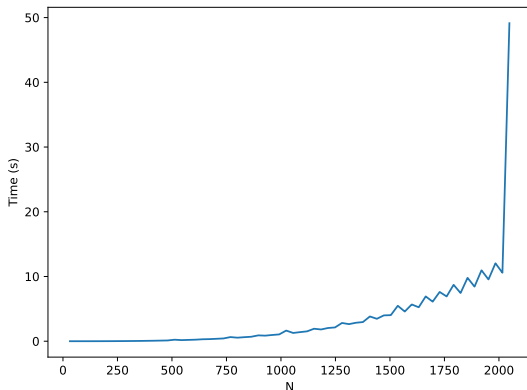
Matrix Multiply

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
  
        float s = 0.0;  
        for (int k = 0; k < N; k++) {  
            s += A[N*i + k] * B[N*k + j];  
        }  
  
        C[N*i + j] = s;  
    }  
}
```

- $C_{i,j} = \sum_{k=1}^N A_{i,k} \times B_{k,j}$
- $O(N^3)$ operation ($2 \times N^3$ float operations)

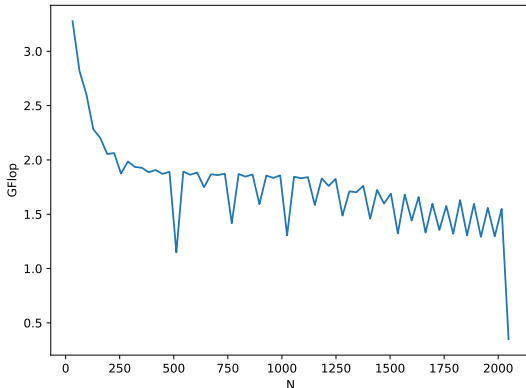
Matrix Multiply : Performance

- Run on an i7 with 128k L1 Cache
- At $N = 2048$
 - $2048 \times 64 \text{ bytes} = 128\text{k}$
 - Next iteration of j will always need to use L2 cache



Matrix Multiply : Sustained GFlops

- Run on an i7 with 128k L1 Cache
- Theoretical peak at around 150 Gflops (single core)
- $\approx 1-2\%$ of Theoretical peak



Matrix Multiply: Outer loop flipping?

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
  
        float s = 0.0;  
        for (int k = 0; k < N; k++) {  
            s += A[N*i + k] * B[N*k + j];  
        }  
  
        C[N*i + j] = s;  
    }  
}
```

```
for (int j = 0; j < N; j++) {  
    for (int i = 0; i < N; i++) {  
  
        float s = 0.0;  
        for (int k = 0; k < N; k++) {  
            s += A[N*i + k] * B[N*k + j];  
        }  
  
        C[N*i + j] = s;  
    }  
}
```

- What is the expected result? Why?

Matrix Multiply : Analysis

Original

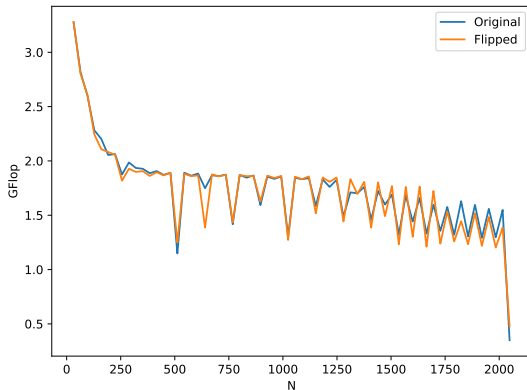
- N^3 reads from A with stride 1
- N^3 reads from B with stride N
- N^2 writes to C with stride 1

Flipped

- N^3 reads from A with stride 1
- N^3 reads from B with stride N
- N^2 writes to C with stride N

Matrix Multiply : Sustained GFlops

- Essentially the same



Matrix Multiply : Transpose

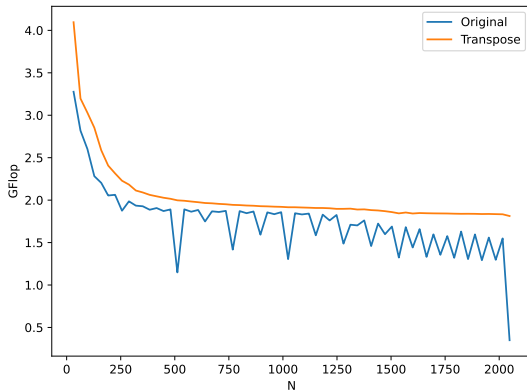
```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
  
        float s = 0.0;  
        for (int k = 0; k < N; k++) {  
            s += A[N*i + k] * B[N*k + j];  
        }  
  
        C[N*i + j] = s;  
    }  
}
```

- In inner loop
 - A stride 1
 - B stride 1
 - C stride 1
- Transpose is $O(N^2)$ and can be improved (previous lecture)

```
float *D = malloc(sizeof(float) * N * N);  
transpose(B, D);  
  
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
  
        float s = 0.0;  
        for (int k = 0; k < N; k++) {  
            s += A[N*i + k] * D[N*j + k];  
        }  
  
        C[N*i + j] = s;  
    }  
}  
free(D);
```

Matrix Multiply : Sustained GFlops

- A modest improvement
- Cache reuse poor



Matrix Multiply : Flipping k loop

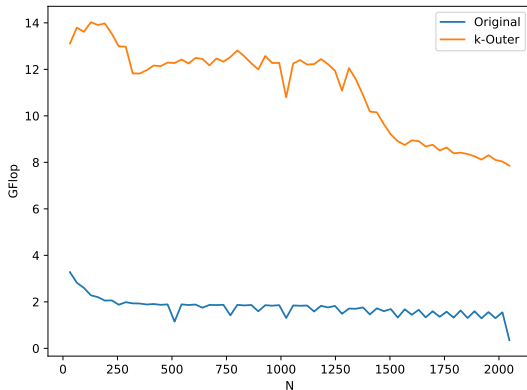
```
for (int i = 0; i < N; i ++) {  
    for (int j = 0; j < N; j ++) {  
  
        float s = 0.0;  
        for (int k = 0; k < N; k ++) {  
            s += A[N*i + k] * B[N*k + j];  
        }  
  
        C[N*i + j] = s;  
    }  
}
```

```
/* Set C to zero */  
for (int k = 0; k < N; k ++) {  
  
    for (int i = 0; i < N; i ++) {  
        for (int j = 0; j < N; j ++) {  
            C[N*i + j] +=  
                A[N*i + k] * B[N*k + j];  
        }  
    }  
}
```

- In inner loop
 - $A[N*i + k]$ constant
 - B stride 1
 - C stride 1
- Initialization is $O(N^2)$ (Can be loop peeled)

Matrix Multiply : Sustained GFlops

- A significant improvement



Matrix Multiply : Transpose vs Flipping k Loop

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
  
        float s = 0.0;  
        for (int k = 0; k < N; k++) {  
            s += A[N*i + k] * D[N*j + k];  
        }  
  
        C[N*i + j] = s;  
    }  
}
```

```
for (int k = 0; k < N; k++) {  
  
    for (int i = 0; i < N; i++) {  
        for (int j = 0; j < N; j++) {  
            C[N*i + j] +=  
                A[N*i + k] * B[N*k + j];  
        }  
    }  
}
```

- Transpose: Read/Write stride 1
- Flipping k: Read/Write stride 1 for C/B
- Cache reuse for A improved (each $A[i][j]$ read once in inner loop)

Matrix Multiply: The case for blocking

- Reads from A and B are $O(N^3)$
- Efficient cache reuse for A and B is challenging

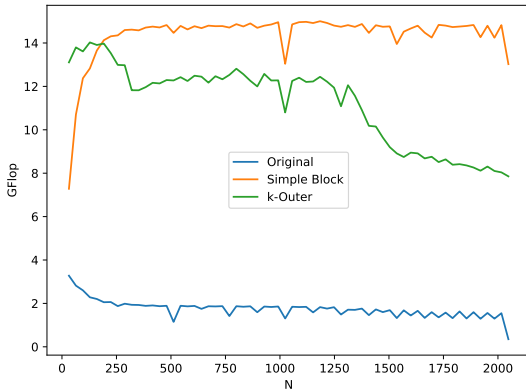
Matrix Multiply: Simple Blocking

```
/* Set C = 0 */
for (block = 0; block < N; block += BLOCK_SIZE) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            float s = 0.0;
            for (int k = 0; k < N; k++) {
                s += A[N*i + k] * B[N*k + j];
            }
            C[N*i + j] = s;
        }
    }
}
```

- Assumes `BLOCK_SIZE` is a factor of `N`
- Aim to choose block size so that inner loops make good reuse of Level 1 Cache

Matrix Multiply : Sustained GFlops

- With Block Size of 16
- Sustained GFlops with increasing N



Matrix Multiply : Optimum Block Size

- Optimum block size is primarily a function of L1 cache size
- End members
 - Block Size = 1 $\rightarrow$ same as k-Outer
 - Block Size = N $\rightarrow$ same as original

```
/* Set C = 0 */
for (block = 0; block < N; block += BLOCK_SIZE) {

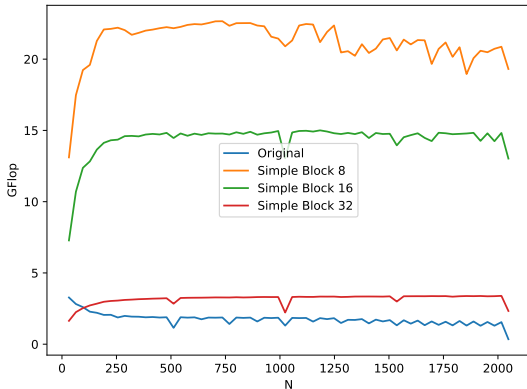
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {

            float s = 0.0;
            for (int k = block; k < block + BLOCK_SIZE; k++) {
                s += A[i*N + k] * B[k*N + j];
            }

            C[i*N + j] += s;
        }
    }
}
```

Matrix Multiply : Sustained GFlops

- Block size of 8 gives better performance



Matrix Multiply : Further optimizations

- Factor of 10 improvement in performance by using blocking size 8
- Improve cache reuse of A and B reads
- Increased number of writes to C
- Remember A/B reads in original are $O(N^3)$ and C writes are $O(N^2)$
- Next step would be efficient implementation using AVX instructions

Matrix Multiply : More

A good tutorial introduction

<https://en.algorithmica.org/hpc/algorithms/matmul/>

Matrix Multiply : More

- OpenBLAS
- <https://github.com/xianyi/OpenBLAS>
- Tuned kernels for many different CPUs

OpenBLAS / kernel / x86_64 / sgemm_kernel_16x4_skylakex.c

Code

Blame

1179 lines (975 loc) · 28.8 KB

```
258     row1 = _mm512_setzero_ps();
259     row2 = _mm512_setzero_ps();
260     row3 = _mm512_setzero_ps();
261     row0b = _mm512_setzero_ps();
262     row1b = _mm512_setzero_ps();
263     row2b = _mm512_setzero_ps();
264     row3b = _mm512_setzero_ps();
265
266     #define KERNEL32x4_SUB()
267     zmm0 = _mm512_loadu_ps(A0);
268     zmm1 = _mm512_loadu_ps(A1);
269     zmm2 = _mm512_broadcastss_ps(_mm_load_ss(B0));
270     zmm3 = _mm512_broadcastss_ps(_mm_load_ss(B0+1));
271     row0 += zmm0 * zmm2;
272     row1 += zmm0 * zmm3;
273     row0b += zmm1 * zmm2;
274     row1b += zmm1 * zmm3;
275     zmm2 = _mm512_broadcastss_ps(_mm_load_ss(B0+2));
276     zmm3 = _mm512_broadcastss_ps(_mm_load_ss(B0+3));
277     row2 += zmm0 * zmm2;
278     row3 += zmm0 * zmm3;
279     row2b += zmm1 * zmm2;
280     row3b += zmm1 * zmm3;
281     B0 += 4;
282     A0 += 16;
283     A1 += 16;
```

Summary

- Matrix multiply is a good example to understand designing algorithms for cache
- Also demonstrates an optimization philosophy: analyse, iterate, verify, repeat
- We've only looked at the simplest possible blocking approach for Matrix Multiply
- More advanced methods consider all levels of cache
- Don't write your own matrix multiply, it is well studied and optimized